

Web Development

What is Web 3.0, and how is it different from Web 1.0 and Web 2.0?

The internet has evolved through different stages, commonly called Web 1.0, Web 2.0, and Web 3.0. Each stage reflects changes in how people use the web and how websites are built.

- Web 1.0 – The “Read-Only” Web

Web 1.0 was the earliest version of the internet, mainly during the 1990s and early 2000s. Websites were static, meaning users could only read information and not interact much with it.

Characteristics:

- Static HTML pages
- Very limited user interaction
- Information published by website owners only
- Few graphics and simple layouts
- Example:
- A company website that only displays information about products and contact details.

- Web 2.0 – The “Read-Write” Web

Web 2.0 introduced user interaction and social networking. Users could now create, share, and comment on content. Most modern websites today are part of Web 2.0.

Characteristics:

- Interactive and dynamic websites
- User-generated content
- Social media and collaboration
- Cloud-based services
- Mobile-friendly applications
- Examples:
- Social media platforms
- Blogs and wikis
- Online shopping websites

- Video-sharing platforms

Web 3.0 – The “Read-Write-Own” Web

Web 3.0 is the next generation of the internet. It focuses on decentralisation, artificial intelligence, blockchain technology, and giving users more control over their data.

Characteristics:

- Decentralised systems using blockchain
- Greater privacy and data ownership
- AI-driven and personalised experiences
- Smart contracts and cryptocurrencies
- Less reliance on large central companies
- Examples:
 - Blockchain-based applications
 - Cryptocurrency platforms
 - Decentralised finance (DeFi)
 - NFT marketplaces

Main Differences Between Web 1.0, Web 2.0, and Web 3.0

Feature	Web 1.0	Web 2.0	Web 3.0
User Role	Read only	Read and write	Read, write, and own
Content Creation	Website owners	Users and companies	Users with decentralised ownership
Interaction	Very limited	Highly interactive	Intelligent and decentralised
Data Control	Website owners	Large tech companies	Users themselves
Technology	Static HTML	Dynamic web apps	Blockchain, AI, smart contracts

2. Functional Differences Between the Front End and Back End of a Web Application

A web application is usually divided into two main parts: the front end and the back end.

Front End

The front end is the part of the website or application that users see and interact with directly in their browser.

Responsibilities:

- Displaying content and layouts
- Handling buttons, forms, and menus
- Creating responsive and interactive designs
- Improving user experience

Common Technologies:

- HTML
- CSS
- JavaScript

Example:

When you click a “Login” button or type into a search bar, you are interacting with the front end.

Back End

The back end works behind the scenes on the server. It processes requests, stores data, and sends information back to the front end.

Responsibilities:

- Managing databases
- Processing business logic
- Authenticating users
- Handling server communication
- Storing and retrieving data

Common Technologies:

- Python
- Java
- PHP
- Node.js
- SQL databases

Example:

When you log into a website, the back end checks whether your username and password are correct.

Key Differences Between Front End and Back End

Feature	Front End	Back End
Purpose	User interface	Server-side processing
Runs On	User's browser	Web server
Focus	Design and interaction	Data and functionality
Technologies	HTML, CSS, JavaScript	Python, Java, PHP, databases
User Visibility	Visible to users	Hidden from users

3. What Happens When You Type a URL Into Your Browser?

When you type a URL into your browser and press Enter, several processes happen before the webpage appears on your screen.

Step-by-Step Process:

1. URL Is Entered

You type a web address such as:

https://www.example.com

The browser first checks whether the address is valid.

2. DNS Lookup

The browser needs to find the IP address of the website because computers communicate using IP addresses, not domain names.

The browser contacts a DNS (Domain Name System) server to translate:

www.hello.com

into something like:

00.111.222.33

3. Browser Connects to the Server

Using the IP address, the browser establishes a connection with the web server using TCP/IP protocols. If HTTPS is used, a secure TLS/SSL connection is also created.

4. HTTP Request Is Sent

The browser sends an HTTP request asking the server for the webpage.

Example HTTP Request:

GET / HTTP/1.1

Host: www.example.com

User-Agent: Mozilla/5.0

Accept: text/html

Explanation:

GET asks for data from the server

/ Means the homepage

Host identifies the website

User-Agent identifies the browser

Accept tells the server what type of content can be received

5. Server Processes the Request

The web server receives the request and may:

Retrieve files

Query a database

Run application code

Authenticate users

The server then prepares a response.

6. HTTP Response Is Returned

The server sends back an HTTP response.

Example HTTP Response:

HTTP/1.1 200 OK

Content-Type: text/html

Content-Length: 123

<html>

<head><title>Example Page</title></head>

<body>

<h1>Hello</h1>

</body>

</html>

Explanation:

200 OK means the request succeeded

Content-Type describes the type of data returned

The HTML content is the webpage itself

7. Browser Renders the Page

The browser reads the HTML, CSS, and JavaScript files and displays the webpage visually for the user.

The browser may also request additional resources such as:

Images

Fonts

Videos

CSS files

JavaScript files

8. Webpage Appears

After all required resources are loaded and processed, the webpage becomes fully visible and interactive.

Summary of the HTTP Request–Response Cycle

1. User enters a URL
2. Browser finds the server using DNS
3. Browser sends an HTTP request
4. Server processes the request
5. Server sends an HTTP response
6. Browser renders and displays the webpage
7. This entire process usually happens within a few seconds.